

Documentacao Tecnica - Escola High Tech Frontend

Versao: 0.2.0 Data de revisao: 2026-05-28

1. Objetivo

Este documento descreve a arquitetura, os modulos, os contratos e os pontos de manutencao do frontend da aplicacao Escola High Tech.

O frontend e uma SPA Nuxt 3 que consome uma API REST externa. Ele controla a experiencia de usuario, roteamento, autenticao no navegador, validacoes locais, permissoes visuais e integracoes de tela. A API continua sendo a fonte final de dados, autorizacao e persistencia.

As funcionalidades ficticias atuais sao QR Code bancario para alunos e holerite demonstrativo para funcionarios. Ambas sao geradas localmente no front e nao possuem valor oficial.

2. Stack tecnica

Tecnologia	Responsabilidade
Nuxt 3.21.6	Framework Vue, roteamento por arquivos, plugins e build
Vue 3	Componentes e Composition API
TypeScript strict	Tipagem de contratos, paginas e utilitarios
Pinia	Estado global, principalmente autenticao
Tailwind CSS	Estilizacao utilitaria
ofetch / \$fetch	Cliente HTTP
@lucide/vue	Iconografia
qrcode	Geracao de QR Code no cliente
Vitest + happy-dom	Testes unitarios
Nginx	Servir build estatico no Docker

3. Configuracao Nuxt

Arquivo principal: `nuxt.config.ts` .

Configuracoes relevantes:

- `ssr: false` : aplicacao gerada como SPA.
- `modules` : `@pinia/nuxt` e `@nuxtjs/tailwindcss` .
- `css` : `~/assets/css/main.css` .
- `runtimeConfig.public.apiBase` : controla a URL base da API.
- `app.baseUrl` : permite deploy em subcaminho.
- `typescript.strict: true` .

Variaveis:

Variavel	Uso
NEXT_PUBLIC_API_BASE	URL publica/base da API
NEXT_APP_BASE_URL	Base URL da aplicacao

4. Estrutura de diretorios

```
assets/css/main.css          # Tailwind e estilos globais
components/                  # Componentes reutilizaveis
docs/                        # Documentacao tecnica
layouts/                     # Layouts default e auth
middleware/auth.global.ts    # Middleware global de autenticacao
middleware/aluno.ts          # Restricao para rotas exclusivas de alunos
middleware/funcionario.ts    # Restricao para rotas exclusivas de funcionarios
pages/                       # Rotas Nuxt
plugins/api.ts               # Plugin que injeta $api
stores/auth.ts               # Store de autenticacao
tests/                       # Testes unitarios
types/api.ts                 # Contratos TypeScript da API
utils/                       # Utilitarios compartilhados
```

5. Autenticacao

Arquivos:

- stores/auth.ts
- middleware/auth.global.ts
- plugins/auth.client.ts
- plugins/api.ts
- utils/api-client.ts

Fluxo:

1. O usuario autentica em `/login` .
2. A resposta de login e persistida no `localStorage` com a chave `form-escola-auth` .
3. O middleware global carrega a sessao, valida via `/auth/me` e protege rotas privadas.
4. O plugin `$api` injeta o token JWT no header `Authorization` .
5. Em resposta `401` , a sessao e encerrada e o usuario volta para `/login` .

Regras do middleware:

- Rotas publicas nao exigem token.
- Usuario autenticado que acessa `/login` volta ao painel.
- Usuario com senha padrao e direcionado a `/alterar-senha` .
- Rotas com `meta.roles` verificam `usuario.descricaoPerfil` .

6. Perfis e permissoes

Utilitario principal: `utils/usuario-permissions.ts` .

Perfis normalizados:

- administrador
- diretoria
- professor
- aluno
- desconhecido

O front usa essas permissões para esconder botoes, campos e acoes. A API deve repetir as regras para segurança real.

7. Modulos de tela

7.1 Login e senha

Rotas:

- /login
- /alterar-senha

Recursos:

- Login via API.
- Reset/esqueci senha.
- Troca obrigatoria quando deveAlterarSenhaPadrao estiver ativo.
- Medidor de força de senha.

7.2 Painel inicial

Rota: / .

Exibe atalhos para:

- Usuarios.
- Caderneta Digital.
- Calendario Escolar.
- QR Code, somente para aluno.
- Holerite, somente para perfis nao alunos.
- Alterar senha.

7.3 Usuarios

Rotas:

- /usuarios
- /usuarios/novo
- /usuarios/:id

Funcionalidades:

- Cadastro e edicao de usuario.
- Busca e filtro por perfil.
- Controle de permissões por perfil.
- Upload e visualizacao de foto.
- Upload, download e exclusao de certificados PDF.
- Popup de contatos.
- Popup de documentos.
- Envio manual de notificacao.

- Campo `dataNascimento` com `DatePicker`.

Campo de aniversario:

- Componente: `components/DatePicker.vue` .
- Entrada digitavel: `dd/mm/aaaa` .
- Valor salvo no estado/payload: `yyyy-mm-dd` .
- Valor vazio enviado como `null` .
- Requer suporte do backend para persistencia definitiva.

7.4 Caderneta Digital

Rota: `/caderneta-digital` .

Funcionalidades:

- Cadastro, edicao e exclusao de disciplinas.
- Lancamento de notas, presencas e faltas.
- Consulta por disciplina.
- Professores administram lancamentos.
- Alunos visualizam apenas registros associados ao proprio cadastro.

7.5 Notificacoes

Implementadas no layout principal.

Funcionalidades:

- Listagem de notificacoes.
- Contador de nao lidas.
- Modal de detalhe.
- Marcar notificacao como lida.
- Marcar todas como lidas.
- Quebra de mensagens longas, URLs de fotos e URLs de PDFs sem overflow horizontal.

7.6 QR Code bancario ficticio

Rota: `/qr-code-bancario` .

Arquivos:

- `pages/qr-code-bancario.vue`
- `middleware/aluno.ts`
- `utils/qr-code-bancario.ts`

Funcionalidades:

- Disponivel somente para alunos.
- Gera dados bancarios demonstrativos por aluno.
- Gera QR Code localmente.
- Compartilha texto por WhatsApp.
- Abre e-mail via `mailto` .
- Copia dados.
- Baixa PNG do QR Code.

Todo payload gerado contem `SEM VALOR BANCARIO` .

7.7 Calendario Escolar

Rota: `/calendario-escolar` .

Arquivos:

- `pages/calendario-escolar.vue`
- `utils/feriados-brasil.ts`
- `utils/date-utils.ts`

Funcionalidades:

- Exibe calendario anual.
- Seleciona o mes vigente por padrao.
- Lista feriados nacionais brasileiros.
- Exibe grade mensal detalhada.
- Permite professor marcar avaliacoes e trabalhos por disciplina.
- Permite editar/excluir eventos.
- Mantem alunos e demais perfis sem permissao de gerenciamento em modo somente leitura.

Persistencia atual:

- A agenda de avaliacoes/trabalhos usa `localStorage` .
- A chave e separada por usuario autenticado.
- Para uso multiusuario real, criar endpoints no backend.
- Para notificar alunos matriculados quando o professor marcar avaliacao ou trabalho, o backend deve relacionar disciplina, matriculas e notificacoes.

Feriados:

- Feriados fixos nacionais.
- Paixao de Cristo calculada a partir da Pascoa.
- Dia Nacional de Zumbi e da Consciencia Negra incluido como feriado nacional.

7.8 Holerite demonstrativo ficticio

Rota: `/holerite` .

Arquivos:

- `pages/holerite.vue`
- `middleware/funcionario.ts`
- `utils/holerite-ficticio.ts`

Funcionalidades:

- Disponivel somente para professor, administrador, diretoria e demais perfis nao alunos.
- Gera contracheque demonstrativo localmente no front.
- Permite selecionar competencia dos ultimos 12 meses.
- Exibe funcionario, cargo, matricula ficticia, proventos, descontos, informativos e valor liquido ficticio.
- Permite copiar resumo textual.
- Permite imprimir a tela pelo navegador.

Rubricas ficticias:

- Salario base.
- Gratificacao pedagogica ou administrativa.

- Auxilio alimentacao.
- Auxilio transporte.
- INSS demonstrativo.
- IRRF demonstrativo.
- Vale transporte.
- Plano de saude.
- Contribuicao associativa.
- FGTS apenas informativo.

Observacoes:

- O demonstrativo sempre informa que nao possui valor trabalhista, fiscal ou contabil.
- Nao ha endpoint de holerite no momento.
- Para holerites oficiais, sera necessario backend com persistencia, autorizacao por usuario, armazenamento/geracao de documento e auditoria.

8. Componentes reutilizaveis

DatePicker

Arquivo: `components/DatePicker.vue` .

Responsabilidades:

- Entrada manual com mascara `dd/mm/aaaa` .
- Conversao para `yyyy-mm-dd` .
- Selecao visual em calendario mensal.
- Navegacao de mes.
- Acoes `Hoje` , `Limpar` e `OK` .
- Estado `disabled` e `required` .

9. Utilitarios

Arquivo	Responsabilidade
<code>utils/api-client.ts</code>	Cliente HTTP e normalizacao de erros
<code>utils/api-file.ts</code>	Download de blobs protegidos por token
<code>utils/api-url.ts</code>	Resolucao de URLs da API
<code>utils/br-phone.ts</code>	Mascara e normalizacao de telefone
<code>utils/caderneta-digital.ts</code>	Parse de notas e regras locais da caderneta
<code>utils/date-utils.ts</code>	Mascara, parse e formatacao de datas
<code>utils/feriados-brasil.ts</code>	Feriados nacionais brasileiros
<code>utils/holerite-ficticio.ts</code>	Rubricas, totais e resumo do holerite ficticio
<code>utils/password-strength.ts</code>	Regras de forza de senha
<code>utils/qr-code-bancario.ts</code>	Dados ficticios e payload de QR Code
<code>utils/usuario-permissions.ts</code>	Regras visuais de permissao

utils/usuario-validation.ts

Validacao de e-mail duplicado

10. Contratos de API

Arquivo: `types/api.ts` .

Principais interfaces:

- `UsuarioSummary`
- `UsuarioForm`
- `UsuarioCreate`
- `UsuarioUpdate`
- `UsuarioArquivo`
- `Notificacao`
- `Perfil`
- `AuthResponse`
- `DisciplinaCaderneta`
- `CadernetaDigitalSummary`

Campo novo:

- `dataNascimento?: string | null` em `usuarios`.

Funcionalidades locais sem contrato de API:

- QR Code bancario ficticio.
- Holerite demonstrativo ficticio.
- Agenda escolar de avaliacoes/trabalhos enquanto persistir em `localStorage` .

11. Endpoints consumidos

Modulo	Endpoints
Auth	POST /auth/login, GET /auth/me, POST /auth/alterar-senha, POST /auth/esqueci-senha
Usuarios	GET /usuarios, GET /usuarios/:id, POST /usuarios, PUT /usuarios/:id, DELETE /usuarios/:id, GET /usuarios/perfis
Arquivos	GET /usuarios/:id/arquivos, GET /usuarios/:id/foto, POST /usuarios/:id/foto, POST /usuarios/:id/certificados, GET /usuarios/:id/arquivos/:arquivoId/download, DELETE /usuarios/:id/arquivos/:arquivoId
Notificacoes	GET /notificacoes, POST /notificacoes, PATCH /notificacoes/:id/lida, PATCH /notificacoes/lidas
Caderneta	GET /caderneta-digital, POST /caderneta-digital, PUT /caderneta-digital/:id, DELETE /caderneta-digital/:id
Disciplinas	GET /caderneta-digital/disciplinas, POST /caderneta-digital/disciplinas, PUT /caderneta-digital/disciplinas/:id, DELETE /caderneta-digital/disciplinas/:id

12. Validacoes

Validacoes locais:

- HTML nativo (`required` , `type` , `maxlength`).
- `utils/br-phone.ts` para telefone.
- `utils/date-utils.ts` para data.
- `utils/password-strength.ts` para senha.
- `utils/usuario-validation.ts` para e-mail duplicado.
- `normalizeApiError` para mensagens vindas da API.

13. Testes

Comando:

```
npm run test
```

Cobertura:

- Store de autenticação.
- Cliente API.
- Telefone brasileiro.
- Caderneta digital.
- Date utils.
- Feriados brasileiros.
- Holerite demonstrativo fictício.
- Força de senha.
- QR Code bancário fictício.
- Permissões de usuário.
- Validação de usuário.

14. Build e deploy

Comandos:

```
npm run typecheck
npm run test
npm run build
npm run generate
```

Deploy estatico:

- Usar `.output/public` .
- Configurar rewrite de SPA para `index.html` .
- Definir `NUXT_PUBLIC_API_BASE` .

Docker:

- Build com Node 22.
- Geracao estatica.
- Servido por `nginx:alpine` .

15. Pendencias e integracoes futuras

1. Persistir `dataNascimento` no backend.
2. Criar endpoints de agenda escolar:

- GET /agenda-escolar
- POST /agenda-escolar
- PUT /agenda-escolar/:id
- DELETE /agenda-escolar/:id

3. Sincronizar agenda entre professores/alunos.
4. Disparar notificacoes para alunos matriculados quando o professor marcar avaliacao ou trabalho.
5. Criar endpoints reais de holerite caso deixe de ser demonstrativo ficticio.
6. Opcionalmente criar envio real de e-mail/WhatsApp para QR Code.
7. Opcionalmente mover feriados para endpoint configuravel, caso haja feriados estaduais/municipais.

16. Manutencao

Ao adicionar novas telas:

- Atualizar `pages/index.vue` se precisar aparecer no painel.
- Atualizar `layouts/default.vue` se precisar titulo dinamico.
- Atualizar `types/api.ts` se houver contrato de API.
- Criar utilitario testavel quando houver regra compartilhada.
- Atualizar README e esta documentacao tecnica.